

Why programs fail?

1. Multithreaded Programs:

- Race Conditions due to improper Synchronization.
- deadlocks causing program to stuck infinitely.
- Livelocks : Thread keep changing state in response to each other.
- Priority Inversion Causing bottlenecks

2. Heap Contention:

- Occurs when multiple threads try to allocate or free memory simultaneously on the heap leading to contention and performance degradation.

3. Heap Leakage:

- Also known as memory leak, occurs when a program allocates memory but never releases it.
- Gradual increase in memory usage, eventually leading to program crashes or system performance issues.

4. Memory Fragmentation:

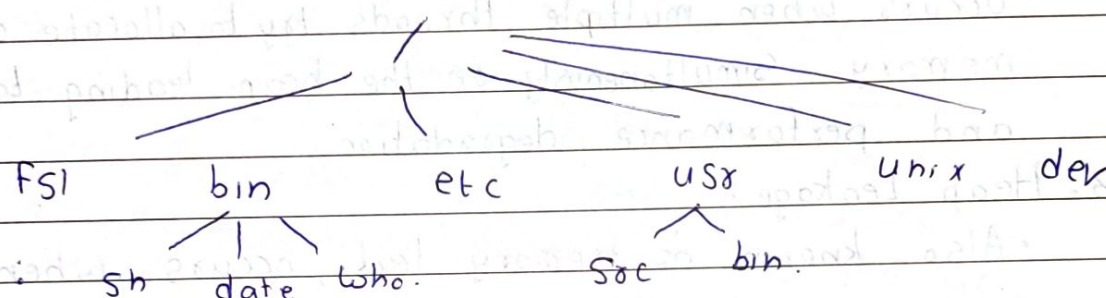
- When memory allocation and deallocation leads to gaps (fragments) in the heap, making it difficult to allocate large contiguous block of memory.

5. False Sharing:

- Occurs when threads on different processors modify variables that share the same cache line, causing excessive cache invalidations and performance degradation.
- Increased CPU cycles due to frequent cache coherency updates.

* Unix File System :

The file system is organized as a tree with a single root node called root (written as "/"). Every non-leaf node of the file system structure is a directory of files, and files at the leaf nodes of the tree are either directories, regular files or special device files.

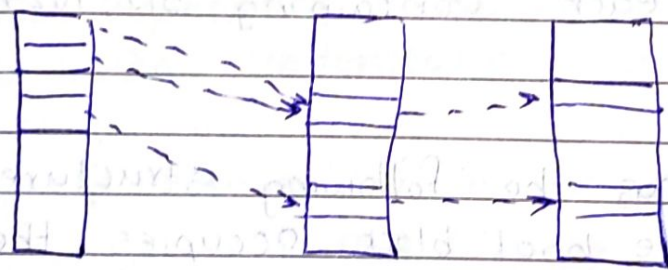


* Kernel Architecture → OS.

* Overview of the file system

- The internal representation of a file is given by an inode, which contains a description of the disk layout of the file data and other information such as the file owner, access permissions, and access times.
- The term inode is a contraction of the term index node and is commonly used in literature on the UNIX system.
- Every file has one inode, but it may have several names, all of which map into the inode.
- Each name is called a link.
- When a process creates a new file, the kernel assigns it an unused inode.

- The kernel contains two other data structures, the file table and the user file descriptor table.
- The file table is a global kernel structure, but the user file descriptor table is allocated per process.
- Entries in three structures - user file descriptor table, file table and inode table maintain the state of the file.
- The file table keeps track of the byte offset in the file where the user's next read or write will start, and the access rights allowed.

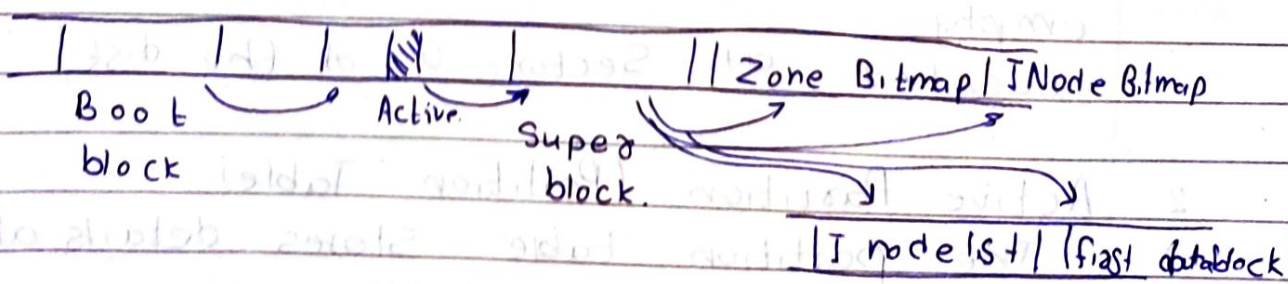


User File
descriptor
table.

File
Table

Inode
table

- The user file descriptor table identifies all open files for a process.



- The kernel returns a file descriptor for the open and create system calls, which is an index into the user file descriptor table.
- When executing read and write system calls, the kernel uses the file descriptor to access the user file descriptor table, follows pointers to the file table and inode table entries and from the inode, finds the data in the file.

- The UNIX system keeps regular files and directories on block devices such as tapes or disks.
- A file system consists of a sequence of logical blocks, each containing 512, 1024, ... etc bytes.

* A file system has the following structure.

- 1) Boot block: The boot block occupies the beginning of a file system, typically the first sector, and may contain the bootstrap code that read into the machine to boot, or initialize, the operating system.
 - It contains boot loader if the system is bootable.
 - It is responsible for loading the OS boot process.
 - If not bootable partition, this section remains empty.
 - Located at Sector 0 of the disk.

2 Active Partition (Partition Table)

- The partition table stores details about different partitions.

- It is located within the master boot record (MBR).

3) Superblock

- The Super block is found immediately after the boot block.
- It is a critical data structure that contains metadata about the file system.
- It includes information such as
 - Size of the file system
 - Number of free blocks and inodes
 - Size of blocks and inodes
 - File System State (clean or dirty)
- The Superblock is essential for the operating system to understand and manage the file system.
- Multiple copies of the Superblock may exist to provide redundancy in case of corruption.
- It has pointers to many other entities like no of data blocks, zone etc.

4) Cluster Zone Bitmaps (Block Bitmaps)

- A bitmap is used to track the allocation status of data blocks in the file system.
- Each bit in the bitmap corresponds to a block in the file system.
 - 1 indicates the block is in use.
 - 0 indicates the block is free.
- It allows the file system to quickly find free blocks for storing data.

5) Inode Bitmaps

- Similar to the block bitmap, the inode bitmap tracks the allocation status of inodes.

- 1 indicates inode in use
- 0 indicates inode is free.

Page : _____

Date : _____

• It helps the file system efficiently manage inode allocation.

6) Inode List:

- Inodes are data structures that store metadata about files and directories.
- Each file or directory has exactly one inode, which contains
 - File type (regular, directory, Symbolic link etc)
 - Permissions (read, write, group etc)
 - Ownership (User ID and group ID)
 - Size of file
 - Time Stamps
 - Pointer to the data block where the file's contents are stored.
- The inode list is an array of all inodes in the file system.
- ~~It~~ Administrators specify the size of the inode list when configuring a file system.
- The kernel references inodes by index into the inode list.
- One inode is the root inode of the file system.
- Root inode is the one by which the directory structure of the file system is accessible after execution of the mount system call.

7) Data Block

- It starts at the end of inode list and contains actual file data and administrative data.
- Files are stored in blocks, and each block can contain a fixed amount of data (eg 4KB).

- * Superblock has pointers to many other entities like
- Total no of data block
 - Total no of zones
 - Total no of zone bitmap blocks
 - Total no of inode (free)
 - Total no of inode bitmap blocks etc.

* 10 Magic Number

- The magic number is a 16-bit value stored in the Superblock of the file system.
- It is used to recognize the type of file system.
- 15 • For Unix File System and its variants (ext2, 3, 4), the magic number is $0xEF53$.
- It is a set of 2-byte (16-bits).
- It plays a critical role in file system operations and integrity checks.
- 20 • File System Identification: The 16-bit magic no uniquely identifies the type of File System (e.g. UFS, ext2, ext3, ext4).
- Error detection: Magic number acts as a quick check for file system corruption. If magic no is missing or incorrect, it indicates that the Superblock may be corrupted.
- 25 • Compatibility: FS utilities (e.g. fsck, mkfs, mount) rely on the magic number to ensure they are working.
- 30 • Recovery: Recovery tools use magic no to identify file system type and attempt repairs.

* Inode :

- An inode (index Node) is a fundamental data structure in Unix file systems.
- It stores meta data about file or directory, excluding its name and actual data content.
- Inodes exist in a static form on disk, and the kernel reads them into an in-core inode to manipulate them.
- Inodes consist of the following fields :
 - i) Mode - Type of file, Permissions.
 - ii) UID, GID
 - iii) Time of last modification
 - iv) Time of last Access.
 - v) Time of last inode changed. (modified).
 - vi) links
 - vii) Pointer to first data block
Pointer to Second data block.
:
Pointer to 10th Data block.
 - viii) Single indirect Pointer
 - ix) Double indirect Pointer
 - x) Triple Indirect pointer.
 - xi) Status of Incore Inode
 - Inode locked
 - A process waiting for inode
 - Incore inode different from disk
 - Incore file different from disk
 - the file is a mount point.
 - xii) Logical device number of file system
 - xiii) Inode Number.
 - xiv) Pointers to other inodes.
 - xv) Reference Count.

ls -i <file names> → get inode no of file.

Page : _____

Date : _____

- An inode is active when a process allocates it, such as when opening a file
- An inode is on the free list only if its reference count is 0, meaning that the kernel can reallocate the in-core inode to another disk inode.
- The free list of inodes thus serves as a cache of inactive inodes.
- If a process attempts to access a file whose inode is not currently in the in-core inode pool, the kernel reallocates an in-core inode from the free list for its use.

15

Lower level File System Algorithms

name ; iget iput bmap	alloc free	ialloc ifree
buffer allocation algorithms		
getblk brelease	bread	breada bwrite

20

Fig. File System Algorithms.

25

- The contents of an inode change when changing the contents of a file or when changing its owner, permission, or link settings.
- The inode lock, when set, prevents other processes from accessing the inode. Other processes set a flag in the inode when attempting to access it to indicate that they should be awakened when the lock is released.

• In Unix inode 1 is typically reserved for the bad block inode. It is used to track bad blocks on the disk.

• Inode 2 is always assigned to the root directory (/).

* Types of files in Unix.

1. Regular Files:

• They are most common types of files.

• They contain data such as text, binary or executable code.

ex. .txt, .c, .jpg, .pdf etc.

• Created using commands like touch, text editors or programs.

2. Directories:

• Special files that act as containers for other files and directories.

• Used to organize the file system hierarchy.

• Ex. /home, /etc, /usr.

• Created using the mkdir command.

3. Symbolic Links (Symlinks):

• Files that act as pointers or shortcuts to other files or directories.

• They contain path to the target file or directory.

• Created using ln -s command.

• ex. A symlink pointing to /usr/bin/python.

4. Device Files:

• Represent hardware devices or virtual devices.

• Located in the /dev directory.

• Two types:

• Character Device Files: Handle data 1 character at a time eg (keyboard, Serial ports).

• Block Device Files: Handle data in blocks.
(e.g hard drives, USB drives).

5. Named Pipes (FIFO Files):

- Special files used for inter-process communication (IPC)
- Data written to a named pipe by one process can be read by another
- Created using the `mkfifo` command.

6. Binary Files:

- It contains data in a binary format, which is not human-readable.
- It stores data in a format that is intended to be processed by a computer program.

15

→ `ln -s /usr/src/kernel S.txt`

Creates a symbolic link named `S.txt` that points to target `/usr/src/kernel`.

20

→ `ln -s /home/user/a.txt /Documents/a.txt`

Creates a shortcut (Symbolic link) of `a.txt` in `home/user` to `documents`.

* 25

`/dev/tty0`: `/dev/tty*` files are special device files that represent terminal devices.

`tyo` stands for first terminal or console.

30

* Where are executable files stored?

→ `dev, bin,/sbin` `/usr/bin`

↓
binaries

↑
System binaries

↑
User level

exe ex. python
gcc.

Camlin

* loopback device: It is a virtual device that allows a file to be treated as a block device.

• This means you can take regular file (eg. ISO) image and mount it as if it were a physical disk.

• It is usually used for working with disk images

ex. power iso.

* What is name of kernel image file.

• bzImage - Big Zipped Image.

• It is compressed linux kernel image format.

• It is stored in /boot and loaded by the boot loader.

* Superblock Structure.

data block Size

15

Total no of data blocks

→ 1kb.

Total no of Zones

Total no of Inode Bitmap block

Total no of Zone Bitmap block

Total no of free Inodes.

20

Total no of Zones (data Block).

First data zone

Maximum file Size.

Magic No Ext2 0xEF53

Pointer to Inode Bitmap.

25

" " " "

Pointer to Zone (Data block

Inode of mounted file System

Inod

Time of last Updation.

30

File System mounted as read only

Dirty flag

- mkfs - Make file System (Used to create a fs on a disk Partition or Storage device .
- fsck - File System Check - Used to Check and repair a F.S. It scans the F.S for inconsistencies.

Q How is link Count different than reference count in inode.

- The link count (also called hard link count) is a field in the inode that indicates how many directory entries (links) point to the inode.
- It link is essentially different name for same file.
- It is stored as i_nlink in inode.
- Reference count (also called as open file descriptor count) tracks how many file descriptors or processes currently have the file open.
- An inode is on the free list when its reference count is 0.

* Visualization of a File / Directory

		/home/mydir/myfile	
/etc ↑ This directory contains the configuration file.	Inode	Name	
	200	.	← mydir
	100	..	← home.
	3000	my file	

when `rm myfile` the inode no is made 00.

* `ls -ll myfile.`

It provides detailed information about myfile

- i - Display inode no
- l : Shows long listing format (permissions, owner, group etc)



- The inode stores metadata about a file and pointers to the data blocks where the file's contents are stored.
- It uses combination of direct, Single indirect, double indirect and triple indirect pointers to manage file data efficiently.

Direct Pointers → Point directly to data block

Single Indirect Pointer → Point to a block that contains more pointers to data block

Double Indirect Pointer → Points to a block containing pointers to Single indirect pointers

Triple indirect Pointer → Point to a block that contains pointer to double indirect blocks

- There are fixed no of direct pointers (ex. 10, 12 (in ext2, ext3))
- Each direct pointer points to a datablock and is used for small files.
- If the file is larger then single indirect pointer is used (file size is below 256 kb)
- If file size is greater than 256 kb then second indirect pointer points to multiple second indirect (double) block.
- Each pointer in second indirect block points to single indirect block

Q. Where from the kernel can get inode no of file from file name.

- directory
- inode list table
- datablock of file.

/proc - Provides details of kernel
df - used to report free disk space.

file Manager of
Linux
Page: _____
Date: Nautilus

* Accessing Inodes.

algorithm iget

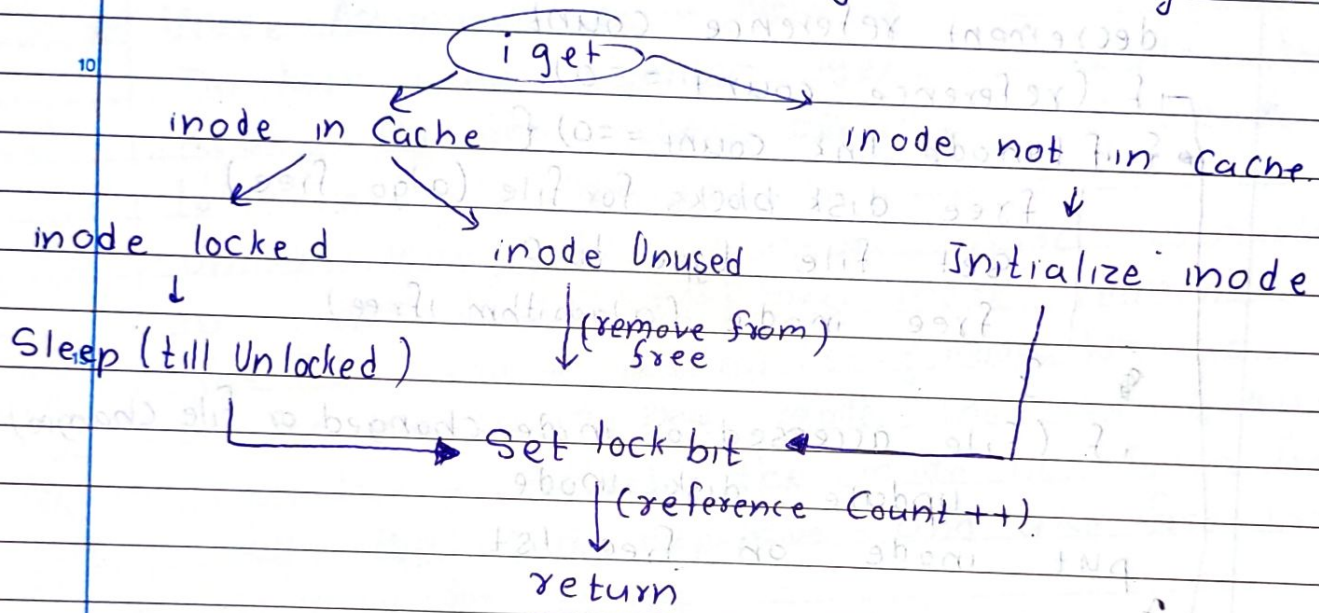
input : file system inode number

Output : locked inode.

```
{  
    while (not done)  
    {  
        if (inode in inode cache) {  
            if (inode locked) {  
                Sleep (wake up when unlocked)  
            }  
            if (inode on inode free list)  
                remove from free list.  
            increment inode reference count.  
            Set lock bit  
            return (inode);  
        }  
        if (no inode on free list)  
            return (error);  
        readinode (algorithm bread)  
        initialize inode (++reference count)  
        Set lock bit.  
        put inode in Inode table.  
        return inode  
    }  
}
```

- Kernel identifies particular inodes by their file system and inode number and allocates in-core inodes.
- The algorithm iget allocates an in-core copy of an inode.

- The kernel maps the device no and inode no into a hash queue and searches the queue for the inode
- If inode is not found, it allocates one from the free list and locks it. Then it prepares to read the disk copy into newly accessed in-core copy of inode (using bread)



- The `iget` algorithm is used toward the beginning of System calls when a process first accesses a file. The algorithm returns a locked inode structure with reference count 1 greater than it already had.

* Releasing Inodes:

- When the kernel releases an inode, it decrements its in-core reference count.
- If the count drops to 0, the kernel writes the inode to disk (if in-core copy is dirty).
- The kernel places the inode on the free inode list.
- The kernel may also release all data blocks associated with the file and free the inode.

if the number of links to the file is 0.

algorithm input

input: pointer to in-core inode

output: none.

{

lock inode if not locked;

decrement reference count

10 if (reference count == 0)

{ if (inode link count == 0) {

free disk blocks for file (algo free)

set file type to 0.

free inode (algorithm ifree)

15 }

if (file accessed or inode changed or file changed)

update disk inode;

put inode on free list.

}

20

release inode lock;

- Input is called while closing a file
- If ref count is not zero after decrementing we just release the inode lock and inode remains in inode table as there is chance of it getting referenced again
- If ^{reference} link count is 0 that means no process is currently using inode, so we put it in unused list.
- 30 If link count is zero we call ifree to remove inode from inode table.

Q What is Setting file type to 0.

The file type is a field in the inode that indicates what kind of file the inode represents

(eg. regular, directory, Symbolic link etc.)

- Setting file type to 0 signifies inode is free and is no longer associated with any file.

Q Inode Assignment to a New file.

- The kernel uses algorithm `iget` to allocate a known inode. Algorithm `ialloc` assigns a disk inode to a newly created file.

- Unlike `iget` where incore inode is allotted in `ialloc` we allot new inode from disk.

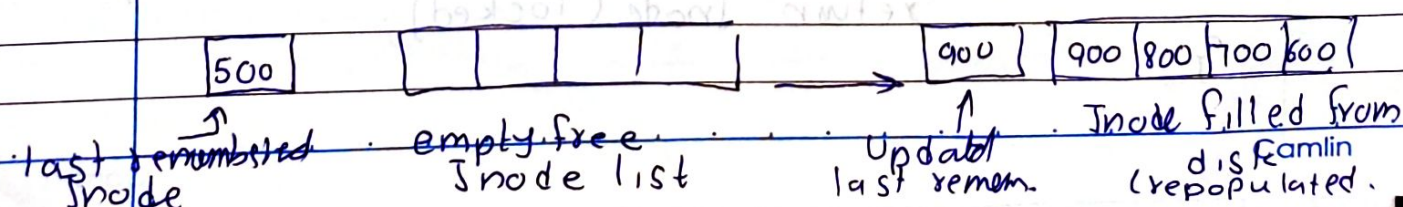
- The FS contains a linear list of inodes. When a process needs a new inode, the kernel could theoretically search the inode list for a free inode. But it is expensive and also free inode list may be empty.

- If free inode list is empty the kernel searches the disk from the last remembered inode till the free inode list is not full.

- Then it remembers the highest-numbered inode that it finds in disk.

- The kernel then initializes and locks inode and returns it.

- The next time when kernel searches the disk for free inodes it uses the last remembered inode as starting point.



Q Where will we Search free inode on the disk.
↳ Search from last remembered inodes as
Starting point.

Page: _____
Date: _____

- Free inode list is present in memory.

Algorithm ralloc.

input - File System.

Output: locked inode

```
{ while (not done) {  
    if (Super block locked) { Sleep (wake up on Unlock) }  
    if (inode (free) list empty).  
    {  
        lock Superblock  
        get remembered inode for free inode Search  
        Search disk for free inode until free  
        inode list is not full. or no more  
        free inodes in disk.  
        unlock Super block  
        if (no free inode found on disk)  
            return (no inode).  
        Set last remembered Inode field.  
    }  
    Get inode from Super block free  
    inode list (algorithm rget)  
    Initialize inode  
        - Zero Inode Buffer content  
        - ++ reference Count.  
        - Put inode in Inod table.  
        - Set lock bit.  
    write inode to disk.  
    -- No of free inodes in Superblocks  
    Make entry in free Inode Bitmap.  
    return Inode (locked);  
}
```


* Freeing an inode

algorithm ifree

input: file System inode number

output: none.

{

increment file System free inode count

if (Super block locked) { Sleep (wakeup on Unlocked) }

if (free Inode list full) {

if (inode $\bar{I}$ < Last Remembered Inode) {

Last Remembered Inode = $\bar{I}$

}

} else {

Store Inode $\bar{I}$ on free list }

Make entry in free inode bitmap

++ Free inodes in Superblock.

return

}

• In ifree the kernel first increments the total no of available inodes in FS.

• The kernel then checks if it has room for more inodes in free inode list

• If free inode list is not full the freed inode is placed there.

• If the list is full the kernel compares the inode number of freed inode with last remembered inode.

• If freed inode no is less than remembered inode, it remembers the newly freed inode number, discarding the old one.

• This inode can be later found from disk.

* Allocation of disk - blocks

algorithm alloc

input : file system number

Output : buffer for new block

{

while (Super block locked) { Sleep (wakeup upon Unlocked)
 remove block from Superblock free list
 if (removed last block from free list)

{

lock Superblock;

read block taken from free list (algo bread);

Copy from buffer to list. (list of free disk block)

release block buffer (Algo - brelease)

Unlock Super block.

wakeup process.

get buffer for block (algorithm getblk);

Zero buffer contents

-- free disk blocks

mark Superblock as modified.

return buffer.

}

• When a process writes data to a file, the kernel must allocate disk blocks from the FS for direct data blocks and sometimes for indirect data blocks.

• The Superblock contains array of free disk blocks in the file system.

• When the kernel wants to allocate a block from file system (algo alloc), it allocates the next available block in the Super block list.

• If the allocated block is the last block in the Super block cache, the kernel treats it as a pointer to a block that contains a list of free blocks.

• It reads the block (bread) and populates the Super block array with new list of blocks and then proceeds with using the original block number.

• It allocates buffer to the block and clears the buffer data.

• It returns the buffer.

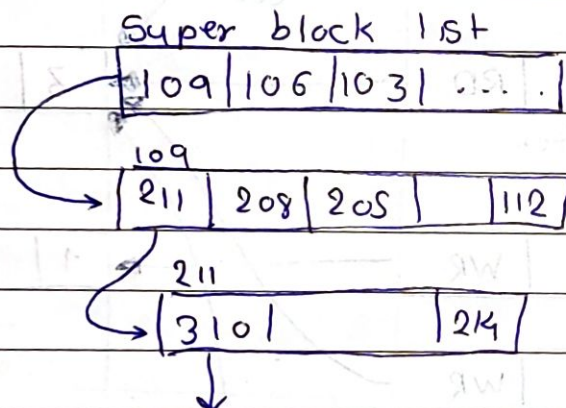


Fig. Linked List of free disk block numbers.

* Freeing a data block:

• It is reverse of the one for allocating a block.

• If the Super block list is not full, the block no of the newly freed block is placed on the Super block list.

• If Super block list is full, the newly freed block becomes a link block, the kernel writes the Super block list into the block and writes block to disk.

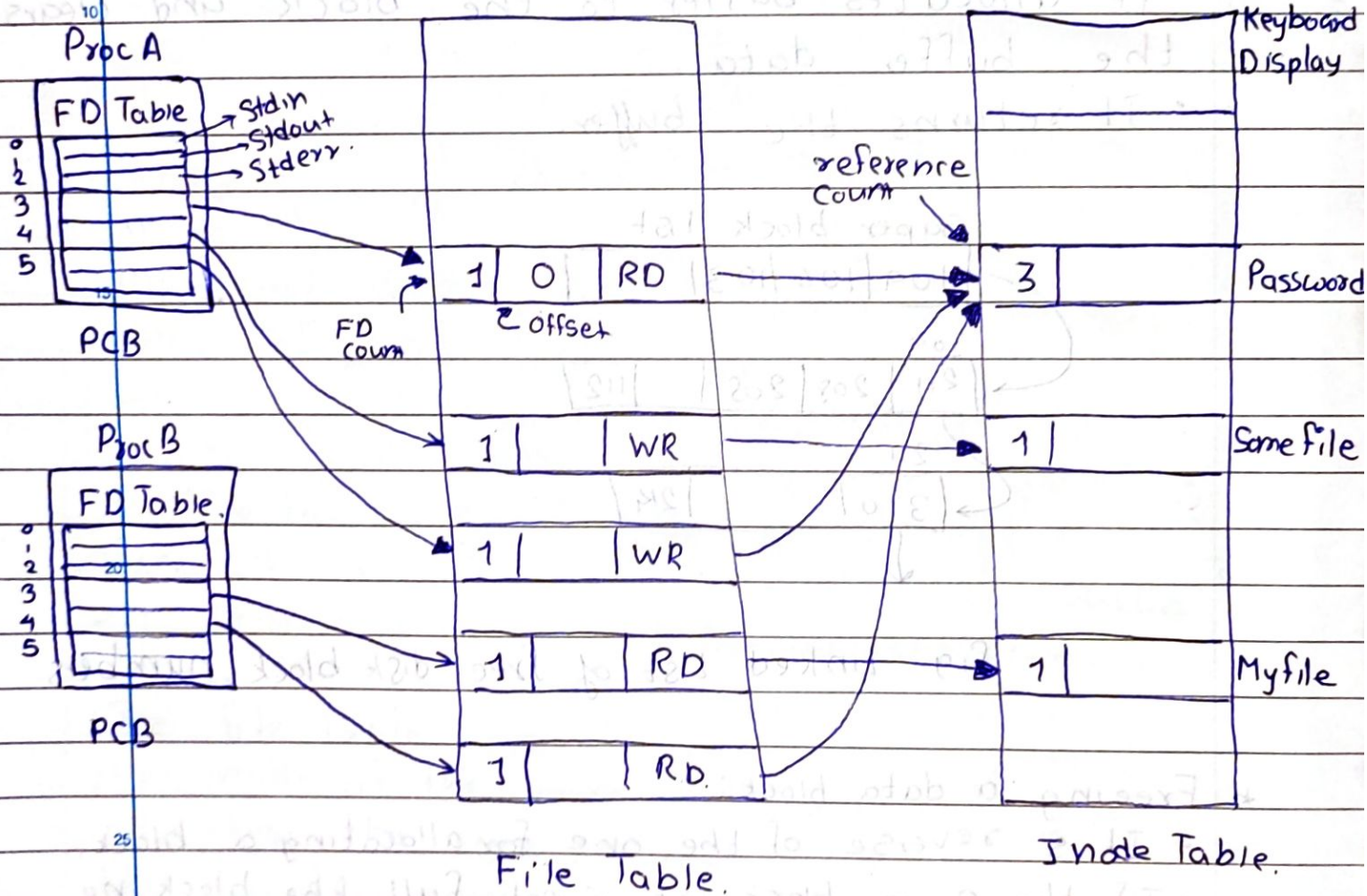
• It places the block number of the newly freed block in the Super block list.

Proc A

```
int fd1 = open("/etc/password", O_RDONLY);
int fd2 = open("Somefile", O_WRONLY);
int fd3 = open("etc/password", O_WRONLY);
```

Proc B

```
int fd1 = open("myfile", O_RDONLY);
int fd2 = open("/etc/password", O_RDONLY);
```



First we create an entry in inode table then we make entry in file table, then we return the file descriptor from FD table (For Creat algo)

System Calls for the File System

Page No.:

Date:

YOUVA

*Open:

The open system call is the first step a process must take to access the data in a file.

• Syntax: `fd = open(pathname, flags, modes);`

- Pathname is file name
- flags indicate type of open (read/write)
- modes give the file permissions if the file is being created.

• The open system call returns an integer called the user file descriptor.

• The kernel searches the file system for the file name parameter using algorithm ~~no~~ name.

• It checks permissions for opening the file after it finds the in-core inode and allocates an entry in the file table for the open file.

• The file table entry contains a pointer to the inode of the open file and a field that indicates the byte offset in the file where the kernel expects the next read or write to begin.

• The kernel initializes the offset to 0 during the open call, meaning that the initial read or write starts at the beginning of a file by default.

• If file is opened in write-append mode the offset is set to size of the file.

All System calls return -1 if they fail.

M T W T F S S

Page No.:

YOUVA

Date:

• The kernel allocates an entry in a private table in the process u area, called the user file descriptor table

Algorithm: open

input: file name

type of open

file permissions

Output: File descriptor

{

Convert file name to inode (algo name)

if (file does not exist or not permit access)

return (error)

allocate file table entry for inode, initialize count

allocate user file descriptor entry, Set ptr to file table

if (type of open specifies truncate file)

Free all file blocks (algo - free)

unlock (inode)

return (user file descriptor)

}

• Each open returns a file descriptor to the process and corresponding entry in the user file descriptor table points to a unique entry in file table

• The file table entries of all instances of an open file point to one entry in the in-core inode table.

• The first three user file descriptors (0, 1 and 2) are called the Standard input, Standard output and Standard error file descriptors.

Processes on Unix Systems Conventionally use the standard input descriptor to read input data, the standard output descriptor to write output data, and the standard error descriptor to write error data.

★ Read:

The Syntax of the read System call is

number = read(fd, buffer, count)

where fd is the file descriptor returned by open, buffer is the address of a data structure in the user process that will contain the read data, count is the number of bytes the user wants to read.

number which is returned is number of bytes actually read.

The kernel gets the file table entry that corresponds to the user file descriptor, following the pointer.

Then it sets several I/O parameters in the u area, eliminating the need to pass them as parameters.

Specially, it sets the I/O mode to indicate that a read is being done.

Then it follows the pointer from file table to the inode, locking the inode before it reads the file.

Then it converts the byte offset into block number to find where to begin read from.

- After reading the block into a buffer it transfers data from the block to the target address in the user process.
- It again updates u area

algorithm read

input : user file descriptor.

: address of buffer in user process.

: number of bytes to read.

Output: Count of bytes Copied into user space.

```

1. get file table entry from user file descriptor.
2. Check file accessibility.
3. Set parameters in u area.
4. get inode from file table.
5. lock inode.
6. Set byte offset in u area.
7. while (count not satisfied)
8. {
9.   convert file offset to disk block (Algo bmap).
10.  calculate block offset and bytes to read.
11.  if (number of bytes to read = 0)
12.    break;
13.  read block (algo bread).
14.  copy data from buffer to user address.
15.  update u area.
16.  release buffer. // locked in bread.
17. }
18. unlock inode.
19. update file table offset for next read.
20. return (total no of bytes read).
21. }

```


* Write

- The Syntax for the write system call is.
number = write (fd, buffer, count);
- The algorithm for writing a regular file is similar to that for reading a file.
- If the file does not contain a block that corresponds to the byte address offset to be written, the kernel allocates a new block using algorithm alloc and assigns the block number to the correct position in the inodes table.

* Close:

- A process closes an open file when it is no longer needed.
- Syntax: close (fd).
fd is the file descriptor of open file.
- If reference count is greater than 1, it is decremented and if it is exact 1 then kernel frees the entry and releases the inode.
- The kernel does close operation by manipulating the file descriptor and corresponding file table and inode table.

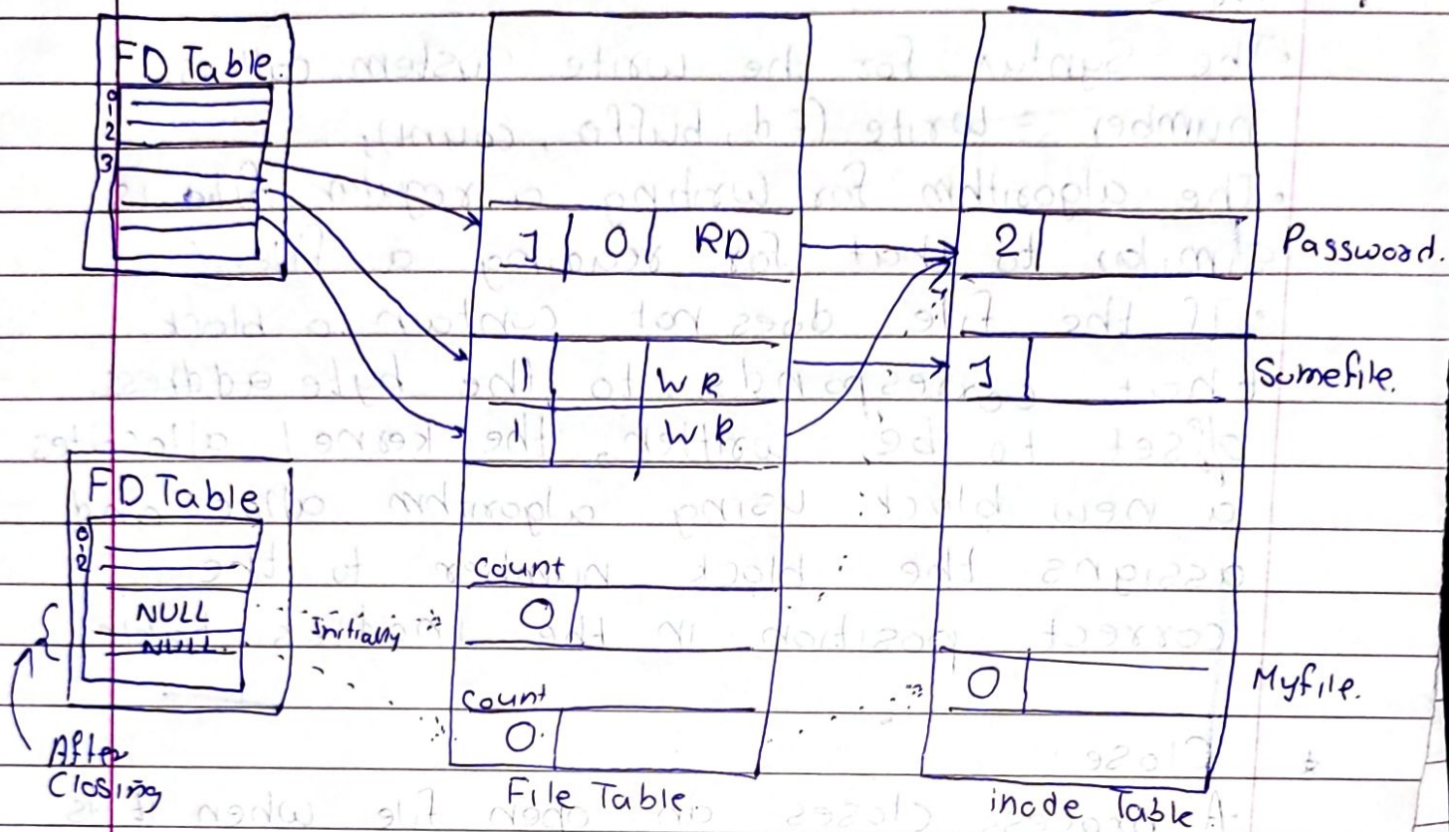
Lets Assume

```

Proc B      int fd1 = open("myfile", O_RDONLY);
            int fd2 = open("/etc/passwd", O_RDONLY);
  
```

was earlier opened.

Now the table after closing second process files will look like.



- The inode reference count is reduce for Password $3 \rightarrow 2$ and for myfile $1 \rightarrow 0$.
- Also the Count field in file table are now 0
- Entries in file descriptor are now empty
- As reference count of myfile is zero it is on free inode list. But its entry is not empty. If some other process accesses myfile the inode is reclaimed.

* File Creation :

The open system call gives a process access to existing file, but the creat system call creates a new file in the system.

Syntax: `fd = creat (pathname, modes);`

- It creates a file if no such file is previously existed and returns its file descriptor.
- New file is created with specified name and permission modes.
- If file was already existed, the kernel truncates the file (releases all existing data blocks and sets the file size to 0).

algorithm creat

input: file name

permission settings

Output: file descriptor

{

get inode for file name (algo frame i);

if (file already exists.)

{

if (not permitted access) {

release inode (algo iput);

return (error);

}

} else //file does not exist.

{ check for write permission in parent directory.

assign free inode from fs (algo ialloc);

create new directory entry in parent dir

Wakeup process

}

allocate entry in file table - FD count

- Permissions.

- File offset

- pointer to inode.
 if (file did not exist at time of Create)
 Free all file blocks (algo free);
 Unlock (disk)
 Set owner group.
 File size $\leftarrow 0$
 Unlock Inode
 Get file descriptor
 point to file table entry.
 return (file descriptor);
 }

- The kernel parse the file path name using algorithm name;
- If no such path name is found a new file is created. after checking the permissions in parent directory.
- New inode is assigned and kernel writes the new file name and inode no in parent director.
- The kernel then writes newly created file to disk (Algo b write)